

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №18
по дисциплине: Основы программирования
тема: **«Обработка строк в стиле C»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2023 г.

Цель работы: решение задач на строки в стиле С.

Содержание отчёта:

- Тема лабораторной работы.
- Цель лабораторной работы.

Вариант 7

Задание 1

```
#include <stdio.h>
#include "string/tasks/string_.h"

//Задание 1
char* getEndOfString(char *s) {
    if (*s != '\0') {
        s++;
    }

    return s;
}

int isgraph(int s) {
    if (s == ' ') {
        return 0;
    }

    return 1;
}

void removeNonLetters(char *s) {
    char *endSource = getEndOfString(s);
    char *destination = copyIf(s, endSource, s, isgraph);
    *destination = '\0';
}
```

Задание 2

```
//Задание 2
void removeAdjacentEqualLetters(char *s) {
    char *c = s;
    char *end = getEndOfString(s);
    while(s != end) {
        s++;
        if (*c == *s) {
            *s = ' ';
        } else {
            c++;
            *c = *s;
        }
    }
}
```

Задание 3

```
int getWord(char *beginSearch, WordDescriptor *word) {
    word->begin = findNonSpace(beginSearch);
    if (*word->begin == '\0') {
        return 0;
    }

    word->end = findSpace(word->begin);

    return 1;
}

void digitToStart(WordDescriptor word) {
    char *endStringBuffer = copy(word.begin, word.end, stringBuffer_);
    char *recPosition = copyIfReverse(endStringBuffer - 1, stringBuffer_ - 1,
word.begin, isdigit);
    copyIf(stringBuffer_, endStringBuffer, recPosition, isalpha);
}

void digitToStartTransform(char *beginString) {
    char *beginSearch = beginString;

    WordDescriptor word;
    while (getWord(beginSearch, &word)) {
        digitToStart(word);

        beginSearch = word.end;
    }
}
```

Задание 4

```
int searchDigit(int s) {
    if (s == 48 || s == 49 || s == 50 || s == 51 || s == 52 || s == 53 || s
== 54 || s == 55 || s == 56 || s == 57 ||
        s == '\0') {
        return 0;
    }

    return 1;
}

char *addSpaces(char *ptr, char *s, char *ptrRead) {
    char buff[MAX_STRING_SIZE + 1];
    memcpy(buff, ptrRead, sizeof(char));
    int value = atoi(buff);

    while (value--) {
        *ptr++ = ' ';
    }

    return ptr;
}

void convertingDigitsToSpaces(char *s) {
    int len_s = strlen(s);
    char *endSource = s + len_s;

    char *buff = copy(s, endSource, stringBuffer_);

    char *recPtr = s;
    char *readPtr = buff - len_s;

    while (readPtr != buff) {
        if (isdigit(*readPtr)) {
            recPtr = addSpaces(recPtr, s, readPtr);
            readPtr++;
        } else {
            memcpy(recPtr, readPtr, sizeof(char));
            readPtr++;
            recPtr++;
        }
    }

    *recPtr = '\0';
}
```

Тесты 4

```
void test_digitToStartTransform1() {
    char c[] = "A32BC1";
    digitToStartTransform(c);
    ASSERT_STRING("123ABC", c);
}

void test_digitToStartTransform2() {
    char c[] = "A32BC1";
    digitToStartTransform(c);
    ASSERT_STRING("123ABC", c);
}

void test_digitToStartTransform3() {
    char c[] = "A32BC1";
    digitToStartTransform(c);
    ASSERT_STRING("123ABC", c);
}

void all_test_digitToStartTransform() {
    test_digitToStartTransform1();
    test_digitToStartTransform2();
    test_digitToStartTransform3();
}
```

Задание 5

```
void replaceNumbersWithSpaces(char *s) {
    char *endStringBuffer = copy(s, getEndOfString(s), stringBuffer_);
    *endStringBuffer = '\0';

    char *readPtr = stringBuffer_;
    char *recPtr = s;

    while (*readPtr != '\0') {
        if (!isdigit(*readPtr)) {
            *recPtr = *readPtr;
            recPtr++;
        } else {
            size_t nSpaces = *readPtr - ZERO_ASCII;
            while (nSpaces--) {
                *recPtr = ' ';
                recPtr++;
            }
        }
        readPtr++;
    }

    *recPtr = '\0';
}
```

Тесты 5

```
void test_replaceNumbersWithSpaces1() {
    char s1[] = "A3B0C1";
    replaceNumbersWithSpaces(s1);
    ASSERT_STRING("A   BC ", s1);
}

void test_replaceNumbersWithSpaces2() {
    char s[] = "3A3";
    replaceNumbersWithSpaces(s);
    ASSERT_STRING("   A   ", s);
}

void test_replaceNumbersWithSpaces3() {
    char s[] = "";
    replaceNumbersWithSpaces(s);
    ASSERT_STRING("", s);
}
```

Задание 6

```
int strcmpWords_(WordDescriptor w1, WordDescriptor w2) {
    while (w1.begin != w1.end && w2.begin != w2.end && *w1.begin ==
*w2.begin) {
        w1.begin++;
        w2.begin++;
    }

    char endWordChar1 = (isspace(*w1.begin)) ? '\0' : *w1.begin;
    char endWordChar2 = (isspace(*w2.begin)) ? '\0' : *w2.begin;

    return endWordChar1 - endWordChar2;
}

void replace(char *source, char *w1, char *w2) {
    size_t w1Size = strlen_(w1);
    size_t w2Size = strlen_(w2);
    WordDescriptor word1 = {w1, w1 + w1Size};
    WordDescriptor word2 = {w2, w2 + w2Size};

    char *readPtr, *recPtr;
    if (w1Size >= w2Size) {
        readPtr = source;
        recPtr = source;
    } else {
        copy(source, getEndOfString(source), stringBuffer_);
        readPtr = stringBuffer_;
        recPtr = source;
    }

    while (*readPtr != '\0') {
        if (!isspace(*readPtr)) {
            WordDescriptor word = {readPtr, NULL};
            getWord(word.begin, &word);
            if (strcmpWords_(word1, word) == 0)
                recPtr = copy(word2.begin, word2.end, recPtr);
            else
                recPtr = copy(word.begin, word.end, recPtr);

            readPtr = word.end;
        } else {
            *recPtr = *readPtr;
            recPtr++;
            readPtr++;
        }
    }

    *recPtr = '\0';
}
```


Тесты 6

```
void test_replace1() {
    char source[] = "ab cm ab ab";
    char w1[] = "ab";
    char w2[] = "d";
    replace(source, w1, w2);
    ASSERT_STRING("d cm d d", source);
}

void test_replace2() {
    char source[] = "cb cm ac";
    char w3[] = "ab";
    char w4[] = "d";
    replace(source, w3, w4);
    ASSERT_STRING("cb cm ac", source);
}

void test_replace3() {
    char source[] = "abdcg cm abdcg";
    char w5[] = "abdcg";
    char w6[] = "d";
    replace(source, w5, w6);
    ASSERT_STRING("d cm d", source);
}
```

Задание 7

```
bool isLexigrOrder(char const *const str) {
    WordDescriptor lastword = {str, NULL};
    if (!getWord(lastword.begin, &lastword))
        return true;

    WordDescriptor word;
    while (getWord(lastword.end, &word)) {
        if (strcmpWords_(lastword, word) > 0)
            return false;

        lastword = word;
    }

    return true;
}
```

Тесты 7

```
void test_isLexigrOrder1() {
    char s1[] = "a b c d";
    assert(isLexigrOrder(s1) == 1);
    fprintf(stderr, "test_isLexigrOrder - OK\n");
}

void test_isLexigrOrder2() {
    char s[] = "b a d c";
    assert(isLexigrOrder(s) == 0);
    fprintf(stderr, "test_isLexigrOrder - OK\n");
}

void test_isLexigrOrder3() {
    char s[] = "";
    assert(isLexigrOrder(s) == 1);
    fprintf(stderr, "test_isLexigrOrder - OK\n");
}
```

Задание 13

```
void getWordsNonEqualLastWord(char *source, char *destination) {
    BagOfWords(_bag, source);
    if (_bag.size == 0) {
        *destination = '\0';
        return;
    }

    size_t lastWordIndex = _bag.size - 1;
    for (int i = 0; i < lastWordIndex; i++) {
        if (strcmpWords(_bag.words[i], _bag.words[lastWordIndex])) {
            destination = copy(_bag.words[i].begin, _bag.words[i].end,
destination);
            *destination = ' ';
            destination++;
        }
    }

    *(destination - 1) = '\0';
}
```